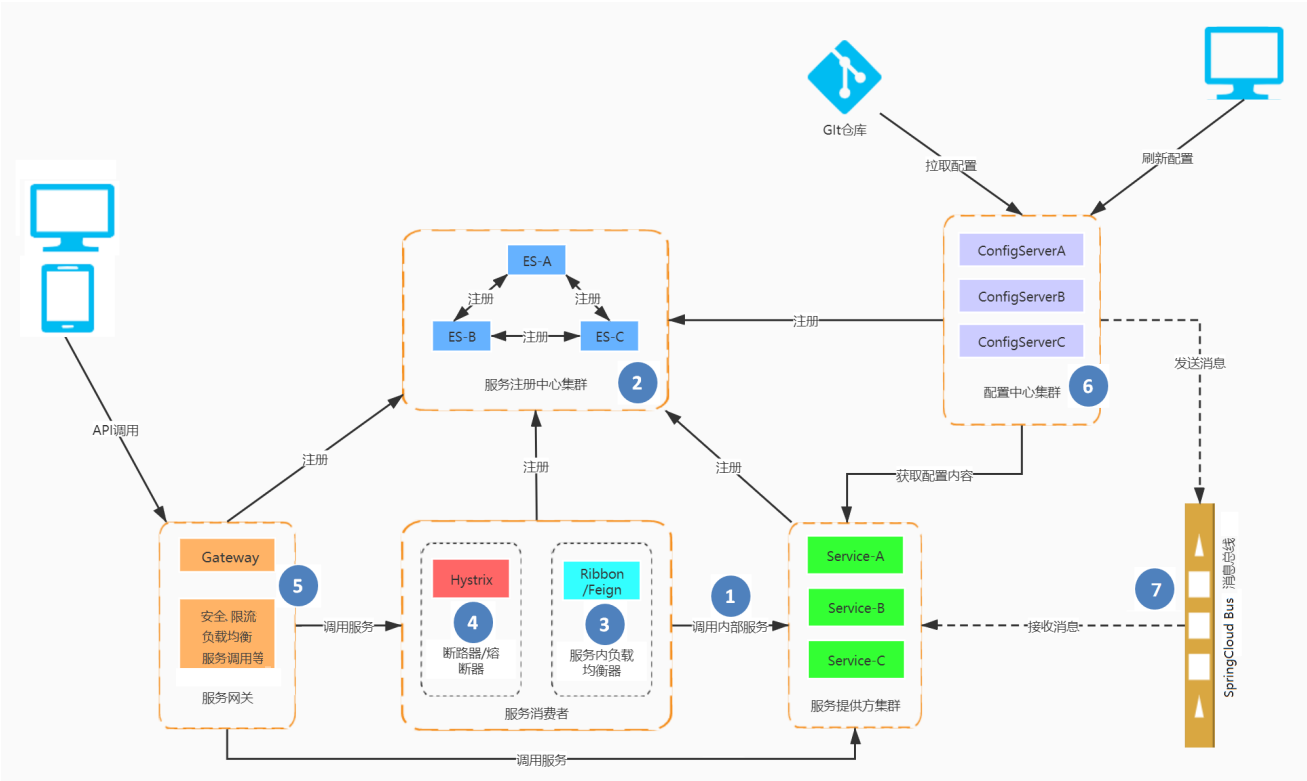
本章所讲内容为 Spring Cloud 架构图中的第二个部分。



1.微服务注册中心

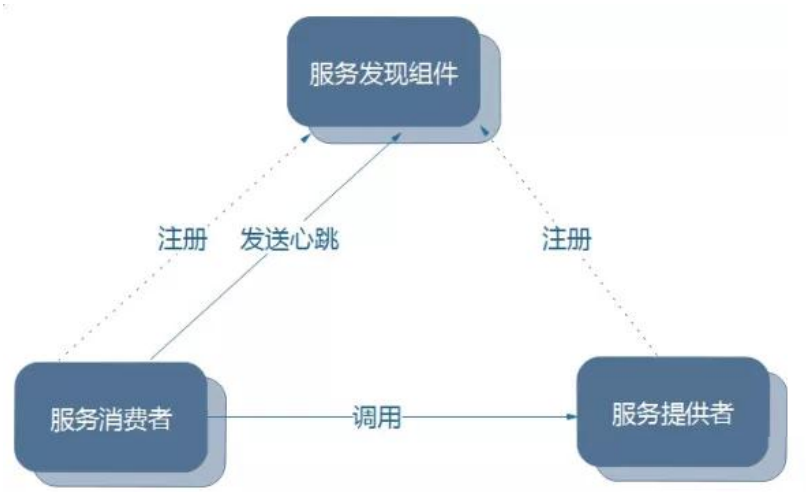
在实际开发中，一个分布式系统中会有成千上万个微服务，此时微服务之间的调用就会出现一些问题。比如：

- 服务消费者调用服务提供者时，请求url中直接写死的。在服务不多时可以这样，但存在大量服务时，手动维护服务列表就会给维护和更新带来很大问题。
- 如果新增微服务时，如何及时通知其他微服务。
- 微服务宕机后，如何及时下线。
- ... ...

微服务注册中心的出现，就是为了更好更方便的管理应用中的每一个服务，是各个分布式节点之间的纽带。

1.1.微服务注册中心原理

服务注册中心主要涉及到三个角色：服务提供者、服务消费者、服务注册中心。



三个角色之间的关系如下：

1. 微服务（包括服务提供者、服务消费者）在启动时，将自己的地址等信息注册到注册中心，注册中心存储这些注册信息。
2. 服务消费者从注册中心查询服务提供者的地址，并通过该地址调用服务提供者的接口。
3. 各个微服务与注册中心使用一定机制（例如心跳）通信。如果注册中心与某微服务长时间无法通信，就会注销该服务实例。
4. 微服务地址发生变化（例如服务增加或IP变动等）时，会重新注册到注册中心。这样，服务消费者就无需人工修改提供者的地址了。

1.2.服务注册中心功能

服务注册中心是微服务架构非常重要的一个组件，在微服务架构里主要起到了协调者的一个作用。注册中心一般包含如下几个功能：

1. 服务注册表
   服务注册表是注册中心的核心，它用来记录各个微服务的信息，例如微服务的名称、IP、端口等。服务注册表提供查询API和管理API，查询API用于查询可用的微服务实例，管理API用于服务的注册与注销。
2. 服务注册与发现
   服务注册是指微服务在启动时，将自己的信息注册到注册中心的过程。服务发现是指查询可用的微服务列表及网络地址的机制。
3. 服务健康检查
   注册中心使用一定的机制定时检测已注册服务的健康状况，如发现某实例长时间无法访问，就会从服务注册表移除该实例。

1.3.注册中心特性对比

目前为止，主流注册中心产品有很多：Nacos（Alibaba）、Eureka、Consul、ZooKeeper等等。

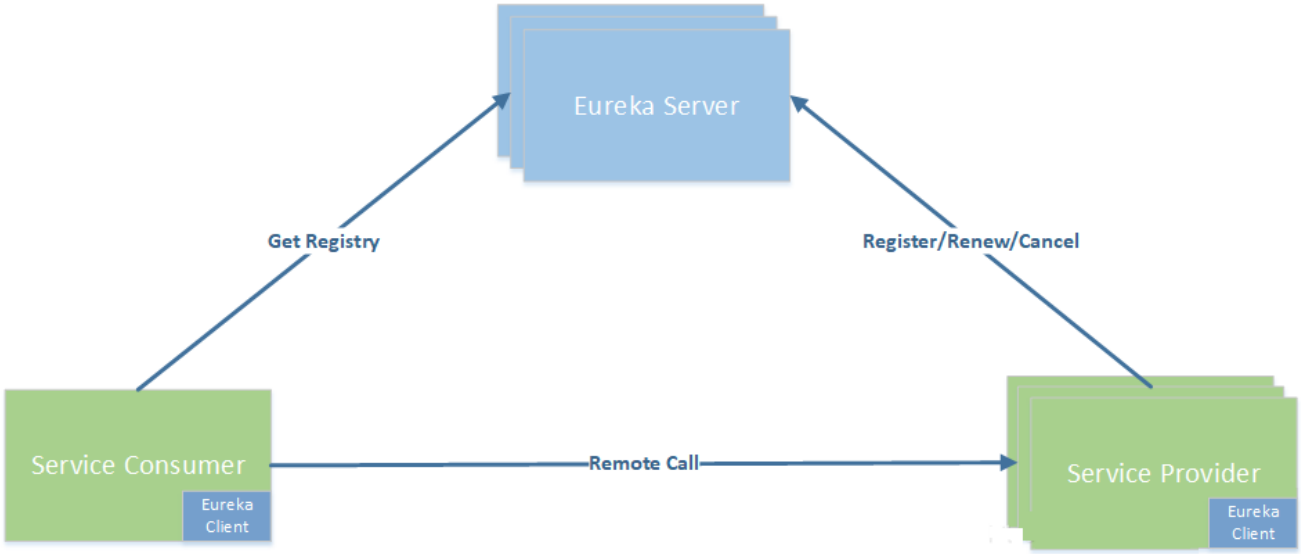
一致性（Consistency）、 可用性（Availability）、分区容错性（Partition tolerance）

比较	nacos	eureka	consul	zookeeper
一致性协议	CP + AP 可切换	AP	CP	CP
健康检查	TCP/HTTP/MySQL/Client Beat	Client Beat	TCP/HTTP/gRPC/CMD	Client Beat
负载均衡	权重/DSL/metadata/CMDB	Ribbon	Fabio	无
雪崩保护	支持	支持	不支持	不支持
自动注销实例	支持	支持	不支持	支持
访问协议	HTTP/DNS/UDP	HTTP	HTTP/DNS	TCP
监听支持	支持	支持	支持	支持
多数据中心	支持	支持	支持	不支持
跨注册中心	支持	不支持	支持	不支持
SpringColud集成	支持	支持	支持	不支持
Dubbo集成	支持	不支持	不支持	支持
kubernates集成	支持	不支持	支持	不支持

2.Eureka服务注册与发现中心

Eureka是Netflix开发的服务发现框架。SpringCloud将它集成在其子项目spring-cloud-netflix中，以实现SpringCloud的服务注册发现功能。

2.1.Eureka基本流程架构



- Eureka基本架构：
  1. Eureka Server：Eureka服务端，提供服务注册和发现
  2. Eureka Client：Eureka客户端，包括服务提供者和服务消费者。
- Eureka基本流程：
  1. 各个微服务启动时，会通过Eureka Client向Eureka Server进行注册自己的信息（例如网络信息），Eureka Server会存储该服务的信息；
  2. 微服务启动后，会周期性地向Eureka Server发送心跳（默认周期为30秒）以续约自己的信息。如果Eureka Server在一定时间内没有接收到某个微服务节点的心跳，Eureka Server将会注销该微服务节点（默认90秒）；
  3. 每个Eureka Server同时也是Eureka Client，多个Eureka Server之间通过复制的方式完成服务注册表的同步；
  4. Eureka Client会缓存Eureka Server中的信息。即使所有的Eureka Server节点都宕掉，服务消费者依然可以使用缓存中的信息找到服务提供者。

综上所述，Eureka通过心跳检测做健康检查和客户端缓存等机制，提高了系统的灵活性、可伸缩性和可用 性。

2.2.Eureka实例

2.2.1.创建Eureka注册中心

在父工程下，创建 Maven Module 子工程（工程名：eureka_server_13000；Packaging：jar）

修改pom.xml文件

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.neusoft</groupId>
```

```
        <artifactId>spring_cloud_demo</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </parent>
    <artifactId>eureka_server_13000</artifactId>

    <dependencies>

        <!-- 添加 eureka server 依赖 -->
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
        </dependency>

        <!--热部署 gav -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

    </dependencies>
</project>
```

> 只需要添加netflix的eureka-server依赖即可创建一个Eureka注册中心

创建主启动类

```
package com.neusoft;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
import org.springframework.context.annotation.Bean;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@EnableEurekaServer  //激活Eureka Server
public class MyApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

> 添加@EnableEurekaServer注解，指定当前工程为Eureka Server端

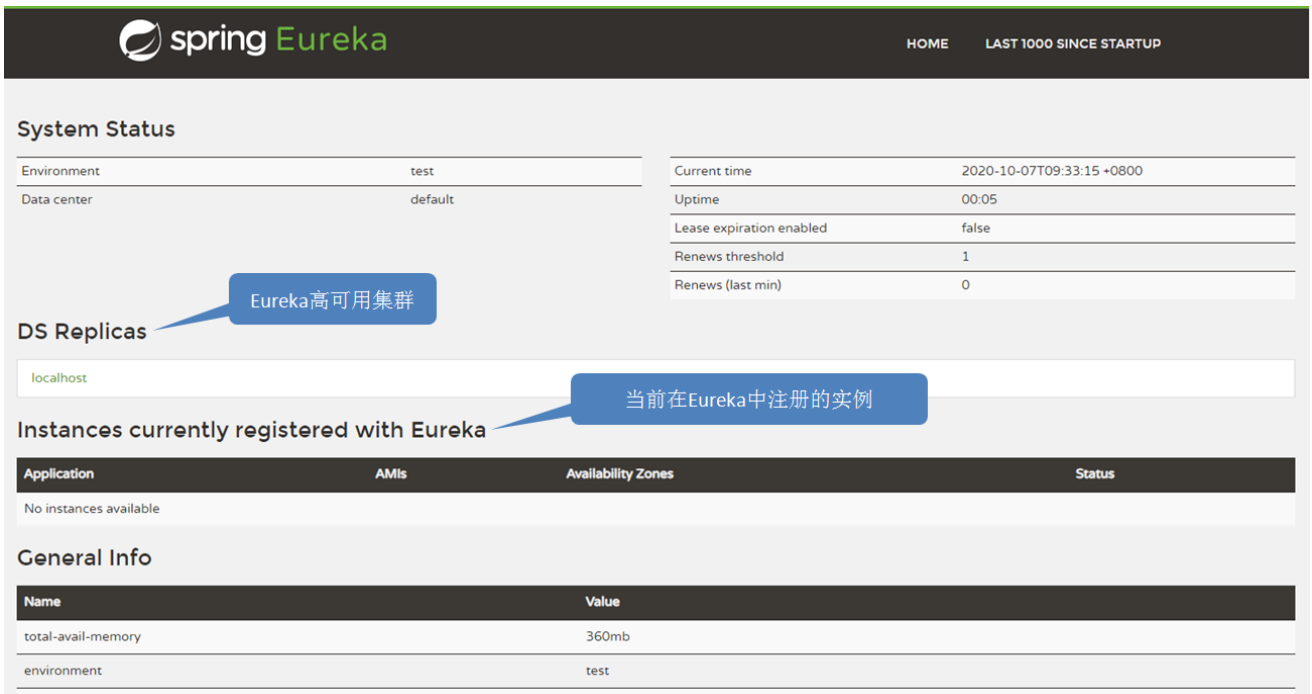
创建application.yml

```yaml
server:
    port: 13000

#eureka配置
eureka:
    instance:
        hostname: localhost   #Eureka Server的主机名（单机版写法）
    client:
        register-with-eureka: false   #是否将自己注册到eureka 服务当中（默认true）.
        fetch-registry: false         #是否启用从注册中心拉取服务列表的功能（默认true）.
        service-url:
            #Eureka Server提供给客户端的访问地址（要加上/eureka/）。
            defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/
```

> - 在Eureka单机模式下，eureka.instance.hostname必须为localhost（或 127.0.0.1）。
> - Eureka Server本身是不需要注册自己的，所以将register-with-eureka与fetch-registry设置为false。

2.2.2.访问Eureka Server管理后台

启动工程，在浏览器地址栏中输入：http://localhost:13000/



2.2.3.服务提供者注册到Eureka Server

修改provider_server_11000工程的pom.xml文件

```
<!--加入eureka clinet的依赖-->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

> 注意： 从SpringCloud的Edgware版本开始，工程中只要添加了此依赖，就成为Eureka Client工程。已经不需要在主启动类上配置@EnableEurekaClient或@EnableDiscoveryClient 注解了。

修改application.yml文件，添加Eureka配置

```
server:
    port: 11000

spring:
    application:
        name: provider-server

#eureka配置
eureka:
    client:
        service-url:
            #需要注册给Eureka Server的地址，也就是将自己注册给哪一个Eureka Server
            defaultZone: http://localhost:13000/eureka
```

重启，在Eureka Server管理界面中就可以看到服务提供者已经注册上去了

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
PROVIDER-SERVER	n/a (1)	(1)	UP (1) - SD-20200611EPOG:provider-server:8001

2.2.4.服务消费者注册到Eureka Server

修改consumer_server_12000工程的pom.xml文件，添加eureka-client依赖。（同上）

修改consumer_server_12000工程的application.yml文件，添加eureka配置。（同上）

启动工程，在Eureka Server管理界面中就可以看到服务消费者也已经注册上去了

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CONSUMER-SERVER	n/a (1)	(1)	UP (1) - SD-20200611EPOG:consumer-server:8888
PROVIDER-SERVER	n/a (1)	(1)	UP (1) - SD-20200611EPOG:provider-server:8001

2.2.5.获取服务列表并调用

服务消费者可以通过Eureka Server获取元数据，通过获取的元数据就可以调用服务提供者了。

Eureka Server元数据有两种：

1. 标准元数据：主机名、IP地址、端口号、状态页和健康检查等信息，这些信息都通过Eureka Server进行获取，用于服务之间的调用。
2. 自定义元数据：可以使用eureka.instance.metadata-map配置，符合KEY/VALUE的存储格式。这些元数据可以在远程客户端中访问。

在程序中可以使用DiscoveryClient接口获取指定微服务的所有元数据信息。

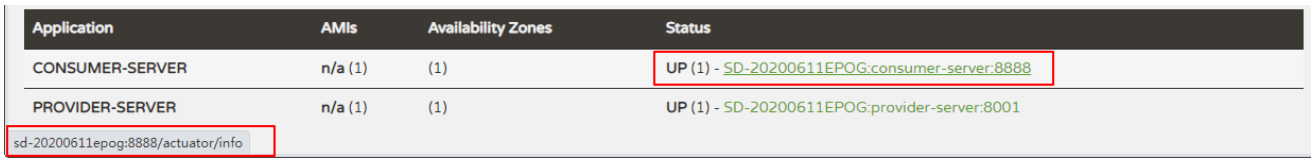
修改consumer_server_12000工程的Controller类，使用DiscoveryClient接口获取元数据，并调用服务提供者。

```
package com.neusoft.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;
import org.springframework.cloud.client.discovery.DiscoveryClient;
import org.springframework.cloud.client.ServiceInstance;

import com.neusoft.po.CommonResult;

@RestController
@RequestMapping("/cart")
public class CartController {

    @Autowired
    private RestTemplate restTemplate;

    @Autowired
    private DiscoveryClient discoveryClient;

    @GetMapping("/getUserById/{userId}")
    public CommonResult getUserById(@PathVariable("userId") Integer userId){

        //通过服务提供者名（provider-server）获取Eureka Server上的元数据
        List<ServiceInstance> instanceList =
                                discoveryClient.getInstances("provider-server");
        //现在，元数据集合中只有一个服务信息
        ServiceInstance instance = instanceList.get(0);

        //使用DiscoveryClient获取元数据，主机地址与端口就可以不硬编码了
        CommonResult result = restTemplate.getForObject("http://"+instance.getHost()+":"+
                  instance.getPort()+"/user/getUserById/"+userId, CommonResult.class);
        return result;
    }
}
```

> 注意：不要导入netflix包下的DiscoveryClient，而是使用SpringCloud提供的DiscoveryClient。

2.3.使用IP和端口向Eureka注册

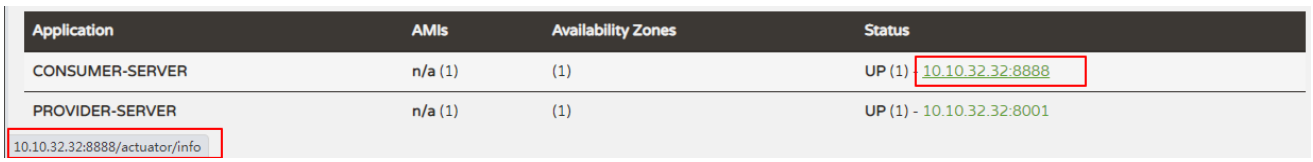
默认情况下，微服务使用主机名向Eureka进行服务注册，以及服务信息的显示； 所以下图中显示的是主机名、服务名和端口。



很显然，使用主机名来注册是不合适的，因为在某些场合下，主机名很有可能出现不能正确解析的问题。所以我们希望使用IP和端口来注册，并且在Eureka管理中心的注册服务列表中的Status列中，显示IP和端口。那么可以在服务提供者和服务消费者的application.yml文件中添加如下配置：

```
#eureka配置
eureka:
    ... ...
    instance:
        #使用ip地址向Eureka注册
        prefer-ip-address: true
        #上面的配置已经可以使用ip注册了，但显示的还是主机名，所以这里设置显示的注册名
        instance-id: ${spring.cloud.client.ip-address}:${server.port}
```

重新启动后，Status列中的微服务就会显示成IP和端口。



2.4.修改续约时间

我们知道，微服务启动后，会周期性地向Eureka Server发送心跳（默认周期为30秒）以续约自己的信息。如 果Eureka Server在一定时间内没有接收到某个微服务节点的心跳，Eureka Server将会注销该微服务节点（默认周期90秒）；

在日志中可以看到续约时间（打开 provider_server_11000日志）：

```
# 启动心跳执行器：续订间隔为：30
Starting heartbeat executor: renew interval is: 30
```

如果认为默认周期时间过长，那么可以通过如下配置来修改续约时间：

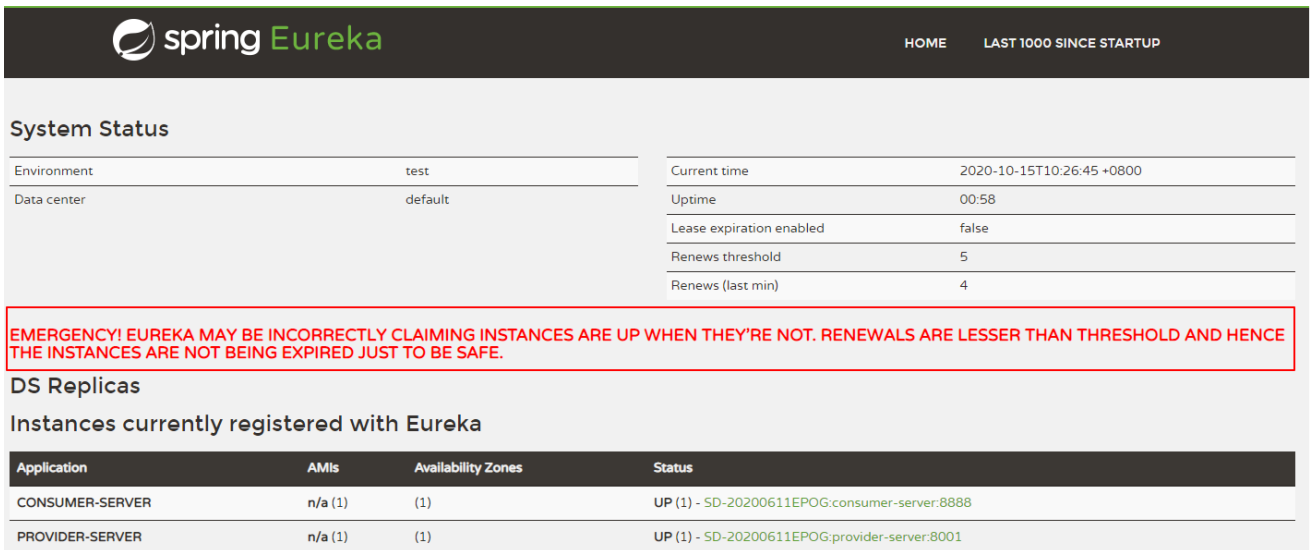
```yaml
eureka:
    ... ...
    instance:
        prefer-ip-address: true        #使用ip地址注册
        instance-id: ${spring.cloud.client.ip-address}:${server.port}
        lease-renewal-interval-in-seconds: 5      #续约时间间隔（秒）
        lease-expiration-duration-in-seconds: 15  #续约到期时间（秒）
```

> - lease-renewal-interval-in-seconds：续约时间间隔（默认30秒）。表示eureka client发送心跳给server端的频率。
> - lease-expiration-duration-in-seconds：续约到期时间（默认90秒）。表示eureka server至上一次收到client的心跳之后，等待下一次心跳的超时时间，在这个时间内若没收到下一次心跳，则将移除该instance。

2.5.Eureka自我保护机制

Eureka自我保护机制：

- 默认情况下，Eureka在90秒内没有接收到某个微服务节点的心跳，就会认为该微服务节点已宕机，并将其从服务列表中剔除。
- 但是，当网络分区故障发生时，微服务与Eureka Server之间无法正常通信，以上行为就会非常危险：因为微服务本身是健康的，此时不应该注销这个微服务。
- Eureka通过自我保护机制来解决这个问题。当Eureka Server的某节点发现在短时间内丢失过多客户端时（可能发生了网络分区故障），Eureka会将当前的实例注册信息保护起来，不在注销任何微服务。
- 具体来说：如果在15分钟内超过85%的客户端节点都没有正常的心跳，那么Eureka就认为客户端与注册中心出现了网络故障，Eureka Server自动进入自我保护机制（在单机状态下很容易进入自我保护机制）。此时，Eureka会将当前的实例注册信息保护起来，同时提示这个警告。
- 当网络故障恢复后，该Eureka Server节点会自动退出自我保护机制。



在开发过程中，我们并不需要自我保护机制。那么可以通过设置eureka.server.enable-self-preservation来关闭自我保护机制。

```yaml
#eureka server配置
eureka:
    client:
        ...
    server:
        enable-self-preservation: false     #关闭自我保护机制
```

重新启动，就会显示自我保护机制已经关闭

```
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.
DS Replicas
```

总结：自我保护模式是一种应对网络异常的安全保护措施。它的架构哲学是：宁可同时保留所有微服务（健康的微服务和不健康的微服务都会保留），也不盲目注销任何健康的微服务。使用自我保护模式，可以让Eureka集群更加的健壮、稳定。

3.Eureka Server 高可用集群

3.1.什么是高可用

高可用（High Availability）是分布式系统架构设计中必须要考虑的因素之一。它通常是指：通过设计减少系统不能提供服务的时间。

我们知道：单节点是系统高可用的大敌，应该在系统设计中避免单节点。高可用保证的原则是“集群化”，或者叫“冗余”。有了集群化，就可以保证在某个节点宕机后，集群中的其它节点能够继续提高服务。

高可用衡量指标：业界用N个9来量化可用性：

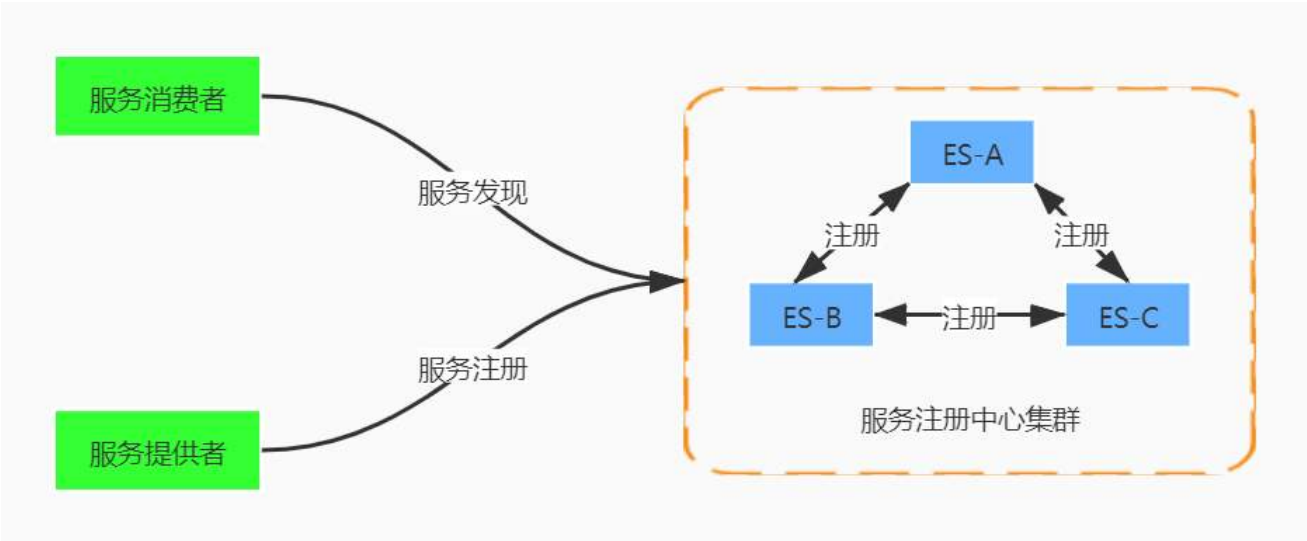
描述	通俗叫法	可用性级别	年度停机时间
基本可用性	2个9	99%	87.6小时
较高可用性	3个9	99.9%	8.8小时
具有故障自动恢复能力的可用性	4个9	99.99%	53分钟
极高可用性	5个9	99.999%	5分钟

3.2.Eureka Server高可用集群

我们已经知道：微服务在获取Eureka Server元数据（服务列表）后会放在本地缓存中。因此一般来说，即使Eureka Server发生宕机，也不会影响到服务之间的调用。

但如果Eureka Server宕机后，某些微服务出现了不可用的情况时，微服务中的服务列表缓存若不被刷新，就可能会影响到微服务的调用，甚至影响到整个应用系统的高可用。

因此，在生产环境中，通常会部署一个高可用的Eureka Server集群。



> Eureka Server可以通过运行多个实例并相互注册的方式实现高可用部署，Eureka Server实例会彼此同步信息，从而确保所有节点数据一致。事实上，节点之间相互注册是Eureka Server的默认行为。

3.3.搭建 Eureka Server高可用集群

3.3.1.修改hosts文件

在一台计算机上搭建Eureka Server集群时，eureka.instance.hostname不能重名，但它又是Eureka Server服务的IP地址，所以需要修改个人电脑中hosts文件。在hosts文件中，就可以将不同的eureka.instance.hostname名映射给同一个本机IP地址。

> Hosts是一个没有扩展名的系统文件，可以用记事本等工具打开。其作用就是将一些常用的网址域名与其对应的IP地址建立一个关联“数据库”，当用户在浏览器中输入一个需要登录的网址时，系统会首先自动从Hosts文件中寻找对应的IP地址。如果没有找到，则系统会将网址提交DNS域名解析服务器进行IP地址的解析。

打开本机的 C:\Windows\System32\drivers\etc\hosts 文件，添加映射信息

```
## springcloud 配置
127.0.0.1 eurekaServer13000
127.0.0.1 eurekaServer13001
```

> 注意：
>
> 1. windows系统下hosts文件内容不区分大小写；Linux系统下区分大小写；
> 2. 不能写localhost，因为localhost是本机域名，不是一个IP地址。

3.3.2.创建Eureka Server集群

既然是创建集群，那么集群中的所有Eureka Server工程，除了IP、端口之外都是一样的。

所以，创建eureka_server_13001子工程。此工程中的内容与eureka_server_13000工程完全相同。

3.3.3.Eureka Server集群的相互注册

修改eureka_server_13000工程中的yml配置文件，添加如下配置属性

```yaml
server:
    port: 13000

#eureka配置
eureka:
    instance:
        hostname: eurekaServer13000   #集群版写法
    client:
        register-with-eureka: false   #是否将自己注册到eureka 服务当中（默认true）.
        fetch-registry: false         #是否启用从注册中心拉取服务列表的功能（默认true）.
        service-url:
            #这是集群版写法，13000注册给13001；反之，13001要注册给13000.
            defaultZone: http://eurekaServer13001:13001/eureka/
    server:
        enable-self-preservation: false     #关闭自我保护机制
```

修改eureka_server_13001工程中的yml配置文件，添加如下配置属性

```yaml
server:
    port: 13001

#eureka配置
eureka:
    instance:
        hostname: eurekaServer13001   #集群版写法
    client:
        register-with-eureka: false   #是否将自己注册到eureka 服务当中（默认true）.
        fetch-registry: false         #是否启用从注册中心拉取服务列表的功能（默认true）.
        service-url:
            #这是集群版写法，13000注册给13001；反之，13001要注册给13000.
            defaultZone: http://eurekaServer13000:13000/eureka/
    server:
        enable-self-preservation: false     #关闭自我保护机制
```

3.3.4.启动服务集群

分别启动两个Eureka Server服务，就可以在高可用集群上分别看到两个服务器。这样，两台服务器互相注册就建立好了。

DS Replicas

eurekaServer7001

3.4.将微服务注册到集群

将微服务注册到Eureka Server集群只需要修改yml配置文件即可。

修改服务提供者provider_server_11000工程的defaultZone配置，添加多个Eureka Server的地址：

```
#eureka配置
eureka:
    client:
        service-url:
            #将自己注册给Eureka Server集群（可以写Eureka服务名，而不必写localhost了）
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
```

修改服务消费者consumer_server_12000工程的defaultZone配置，添加多个Eureka Server的地址：

```
#eureka配置
eureka:
    client:
        service-url:
            #将自己注册给Eureka Server集群
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
```

> 注意：
>
> 1. 实际上，由于Eureka Server集群的各个节点相互注册后，服务只需要注册到集群中的某一个节点上，就相当于注册到集群中的所有节点上。
> 2. 但是，当服务只注册到集群中的某一个节点上时，如果这个节点宕机了，那么该服务将无法注册。所以，实际开发中，都会将服务注册到集群中的所有节点上。

4.Eureka 核心工作流程总结

下面对Eureka 核心工作流程做一个总结：

1. eureka server 启动成功，服务端注册，在启动过程中如果配置了集群，集群之间定时同步注册表，每个eureka server都存在独立完整的服务注册表信息。
2. eureka client启动时根据配置的eureka server地址去注册中心注册服务。
3. eureka client会每30S 向eureka server发送一次心跳请求，证明客户端服务正常在线。

4. 当eureka server 90S内没有收到eureka client的心跳，注册中心会认为该client节点失效，会注销该实例。
5. 在单位时间内，eureka server统计到有大量的client没有发送心跳，则认为为网络异常，进入自我保护机制，不再剔除没有发送心跳的客户端。
6. 当client心跳请求恢复正常以后，eureka server 自动退出自我保护模式。
7. client定时全量或增量从注册中心获取服务注册表，并且将获取到的信息缓存到本地。
8. 服务调用时，client会先从本地缓存中找调取的服务，如果获取不到，先从注册中心刷新注册表，再同步到client的本地缓存。
9. eureka client获取到目标服务器的信息，发起服务调用。
10. eureka client关闭时向eureka server发送取消请求，eureka server将client实例从注册表中删除。